Software

Software Overview

Engaging has its own software stack. Many basic and commonly used software and libraries are already installed, so it is good to check before spending the time to install it yourself. This page discusses the general overview of what kinds of software are supported and points you to how to use them and what to do if what you need isn't there.

Software Landscape

While the software stack will be different on each system, there are three general classes of software:

Category	Description
Core	Commonly used or fundamental software and libraries that are fully supported. Core software is expected to work until it is officially deprecated, and often newer versions are provided to replace them.
Community	Software that has been built and installed by request, but is not commonly used. Support is on a best-effort basis. These should work when built but are not guaranteed to work indefinitely or when replaced with newer versions when deprecated, except by request.
Deprecated	Software that is no longer supported or expected to work. May be kept for legacy reasons, or will soon be removed. If software you are using is listed as deprecated or soon to be deprecated, migrate to the newer version (if available) or request a newer version (if not available). If migrating to a newer version is not an option you may be able to run your application with Apptainer.

Some older nodes on Engaging have a different software stack with older software versions. These nodes run an older operating system (CentOS 7). Software built for the older operating system is not expected to work on the newer nodes and is no longer supported.

Steps for Getting Software

One of the first steps for getting a workflow running on a new system is to set up any software or packages needed to run it. Here are a few steps to do that on Engaging.

Check if the Software or Package is Already Installed

As mentioned above, there is a lot of software already installed. Using the software we've installed saves you time. This software may also perform better or be better configured to use on the system. For example, it may be installed in a faster part of the filesystem or configured to use special hardware available on the cluster.

For software check the `module avail` command (see the page on modules for more information). Some software is available without a module, you can check if a particular command is available using the `which` command at the command line. For example, run `which git` to see if the `git` command is available. If it is, the path to the `git` command will print to the screen.

Common languages like Python, Julia, and R are provided through modules as well. Packages for these are sometimes provided along with the installation. A quick way to check if a package is available is to try to import it.

Install the Software or Package

If we don't have the software you need, you can often install it yourself. You will need to install them in your home directory or another directory you have access to. You will not be able to install software in any of the system-wide directories, as changes to these affect everyone using the system (for example you will not be able to install in any location that requires `sudo`).

Why can't I use sudo?

The `sudo` command is used to make system-wide administrator-level changes. On a system where you are the only user this is usually fine, the only person you can affect is yourself. On large shared systems with many users any command that uses `sudo` has the potential to affect the workflow of other researchers and potentially cause harm, even when it is not intentional. For this reason only trained system administrators have the ability to use `sudo`.

You should not need `sudo` to install packages in your own space. For software installs, the `sudo` is only used to put the installation files in the system-wide directory, so it is not needed to install in your own directories. The Installing Software page covers how to specify installation directories for some of the more common build systems.

Sometimes you can find pre-built binaries for the software you want. These are the easiest to install. Often you will need to build the software you need. See the page on Installing Software for more information. You may also check the Recipes section of these pages to see if there is an existing recipe for installing the software you are interested in.

For Python, Julia, and R packages, each of these have their own package managers for installing packages. See the respective documentation pages linked above for each of these.

Ask for Help

If you are having trouble installing software you can reach out to orcd-help@mit.edu or one of the other lists on Getting Help for help. You can also stop by office hours if you prefer. Depending on the software and the system you are using, we may help walk you through installing it for yourself or install it in a community location.

February 11, 2026